

When Cheap Transformations Vanish: A Diagnostic Study of Shallow Symbolic Solvers on ARC-AGI-2

提交前必须替换的 3 处占位符（否则 **Writeup** 不完整）

1. [repository URL] → 你的公开 GitHub 仓库地址

2. [Kaggle notebook URL] → 公开 notebook 地址（Project Links 字段）

3. [submission ID] → ARC-AGI-2 代码提交的 submission ID

另需在 Media Gallery 上传两张图：cover.png（封面，必需）与 coverage_curve.png。

Subtitle

Coverage, not search, governs ARC-AGI-2 for fixed-vocabulary solvers: we quantify a transformation-family shift, show coverage collapses to a cliff rather than a slope, and recover +44% relative by spending the second attempt on a different hypothesis class.

Track

Paper Track (the same Writeup is also submitted as the ARC-AGI-2 Solution Writeup; the two tracks share one six-dimension rubric).

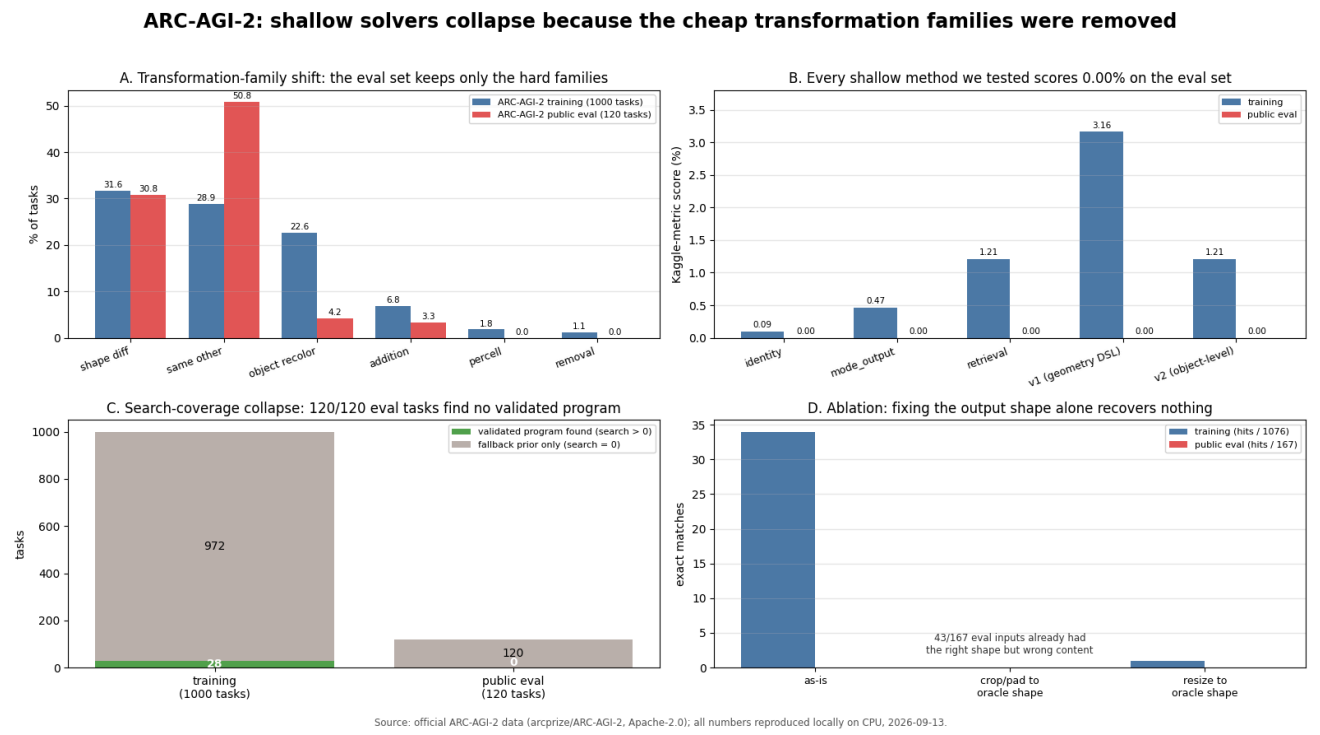


Figure 1 — Family shift, method scores, coverage collapse, and the shape ablation.

1. Claim

Shallow symbolic solvers—program search over a fixed vocabulary of grid transformations—are a standard ARC baseline, yet they score near zero on ARC-AGI-2. This is not an implementation artifact. We introduce a reproducible transformation-family taxonomy, measure a sharp distribution shift between the two public splits, show that the dominant eval family (50.8%) needs object-level conditionals our vocabulary cannot express, and show by controlled ablation that output-shape inference—the usual suspect—is not the bottleneck: correcting shapes recovers exactly zero points.

2. Setup

One discipline, four search spaces. A candidate is accepted only if it reproduces *every* demonstration exactly; parameters (colour maps, per-property palettes, fill colours) are fitted on the demonstrations, never guessed.

- **A — geometry DSL.** 29 primitives (dihedral transforms, crops, gravities, scaling, tiling, mirror completion, uniform row/column removal, extreme-component crops) composed to depth ≤ 2 : ~318 chains per task. Training coverage 2.7%.
- **B — object-level operators.** Components with properties (size, colour, bbox, border contact, squareness, holes); predicates select objects, actions delete, keep or recolour them. Training coverage 1.9%.
- **C — context-conditioned recolouring.** Cells are described by structural features (own colour, its object's size and size-rank, boundary/border/hole/square flags); the map feature-tuple $\rightarrow$ colour is fitted on the demonstrations. Training coverage 3.8%, 4.3% composed behind an object filter.

Metric and data. Kaggle's metric: two attempts per test input, exact match on either scores 1, final score is the mean. Data: the official ARC-AGI-2 release (1,000 training, 120 public eval tasks). Nothing is trained and no eval feedback was used for tuning.

3. Diagnosis

3.1 Family taxonomy and distribution shift

We label every demonstration pair by the simplest family that explains it (shape-changing; per-cell colour function; uniform object recolouring; pixel removal; pixel addition; otherwise object-level conditional), and give each task its dominant label ($\geq 80\%$).

Family	Training (1000 tasks)	Public eval (120 tasks)
shape-changing	31.6%	30.8%
object-level conditional (F_same_other)	28.9%	50.8%
uniform object recolouring	22.6%	4.2%
per-cell colour function	1.8%	0.0%
pure removal	1.1%	0.0%

Structural probes agree: outputs that are integer tilings of the input fall from 0.3% to **0.0%**, contiguous sub-grids from 5.2% to **0.0%**, tiny counting outputs (≤ 4 cells) from 2.1% to **0.0%**. Eval keeps the hard families and drops precisely those our vocabulary covers cheaply.

3.2 Coverage collapse, and why it is a cliff rather than a slope

Coverage collapses as a cliff, not a slope: the vocabulary is orthogonal to ARC-AGI-2 eval

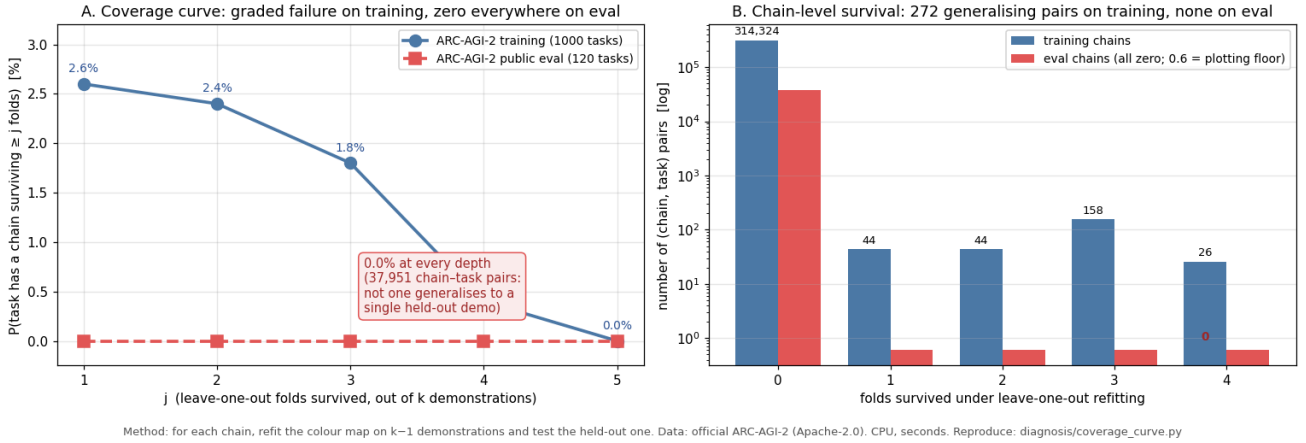


Figure 2 — Coverage curve: graded failure on training, identically zero at every depth on eval.

Coverage—the fraction of tasks admitting at least one verified program—collapses to zero:

	training	public eval
program reproducing every demonstration	27 / 1000	0 / 120
program surviving leave-one-out refitting on ≥ 1 fold	26 / 1000	0 / 120

Binary coverage hides the shape of the failure. Refitting each chain on $k-1$ demonstrations and testing the held-out one yields a *coverage curve*. On training it decays gradually (2.6% $\rightarrow$ 2.4% $\rightarrow$ 1.8% $\rightarrow$ 0.4% $\rightarrow$ 0%), so the vocabulary sits near the boundary for many tasks. On eval the curve is **identically zero at every depth** (Figure 2): across 37,951 chain–task pairs, not one chain generalises to even a single held-out demonstration on any of the 120 tasks. Nor is this one design's artefact: all four search spaces we built—geometric (2.7% training coverage), object-level (1.9%), context-conditioned (3.8%), and depth-2 compositions of object filters with context recolouring (4.3%)—validate on **exactly 0/120** eval tasks while being structurally unrelated. The vocabulary is not slightly too small for eval; it is orthogonal to it. Score follows: **4.55%** on training, **0.00%** on eval.

3.3 Shape inference is not the bottleneck

Shape prediction is the usual diagnosis (74% of eval failures). We tested it:

- **Shape-predictability.** "Output equals input shape" matches **71.3%** of eval test inputs, non-background bounding box 56.9%, our fallback's rule (modal demonstration shape) only 25.1%—measurably suboptimal.
- **Controlled ablation.** Cropping/padding predictions to the oracle shape yields **0/167**; nearest-neighbour resizing to it also **0/167**. 43/167 eval inputs already had the right shape and still missed on content.

Shape error is therefore a *symptom* of falling back to priors, not an independent defect.

3.4 A floor of trivial baselines

Demonstration-only baselines: identity (0.09% train / 0.0% eval), modal output (0.47%/0.0%), retrieval (1.21%/0.0%), per-cell majority (0.56%/0.0%). A beats them all on training—the search does real work—yet **every shallow method is exactly 0.00% on eval**.

4. Intervention: spending the second attempt on a different hypothesis class

ARC-AGI-2 grants exactly two attempts per test input. Let A have hit set H_A and B have H_B . Spending both attempts inside A realises at most H_A ; spending the second on B realises $H_A \cup (H_B \setminus H_A)$. **The return on the second slot is therefore exactly the asymmetric difference $|H_B \setminus H_A|$ —not B's accuracy.** A weaker but disjoint solver can beat a stronger but redundant one.

Allocation of the two attempts	Training hits	Score
A attempt_1 only	32 / 1076	2.974%
A attempt_1 + A attempt_2 (same class)	34 / 1076	3.160%
A attempt_1 + B attempt_1 (different class)	45 / 1076	4.182%
A attempt_1 + C attempt_1 (different class)	49 / 1076	4.554%

Here A is the geometry DSL, B the object-level solver and C the context-conditioned solver. The heterogeneous slot returns **+1.02 pp** against **+0.19 pp** for a second guess from the same solver—5.5× per slot—and the strongest complementary class (C) lifts the portfolio to **+1.39 pp / +44% relative**. Gains are localised, not diffuse:

Family	n	A	A+B	A+C
uniform object recolouring	232	1	11	12
object-level conditional	314	7	8	11
shape-changing	353	18	18	18
per-cell / addition / removal / mixed	177	8	8	8
total	1076	34	45	49

Every added hit lands in the two object families—the only families where B and C are strong—and **no family regresses**, exactly as the set-difference reading predicts. The transferable rule for anyone working under a fixed attempt budget: measure your solvers' hit sets, then spend the second slot to maximise the difference—not to sample the same model twice.

5. Interpretation: why coverage, not search, governs this distribution

Fix a vocabulary V and a depth budget k ; let $\text{Programs}_{\leq k}(V)$ be what it can express. Define coverage $C(V) = P_{t \sim D}[\exists p \in \text{Programs}_{\leq k}(V) \text{ that reproduces every demonstration of } t]$. Score is bounded by coverage: a test input whose demonstrations admit no expressible program can only be answered by luck. We argue $C(V)$ is governed by *family alignment*, through two mechanisms.

Mechanism 1—family removal. A finite vocabulary induces a set of expressible transformation families $F(V)$. When the evaluation distribution is constructed to exclude the families that $F(V)$ covers cheaply, those primitives become dead weight and coverage falls to zero rather than merely degrading. Per-cell recolouring, tiling, sub-grid extraction and counting are 1.8–5.2% of training and **0.0%** of eval, where coverage is **0/120**—not low, but zero.

Mechanism 2—compositional explosion. Object-conditional tasks are described by a predicate over objects, an action, and optionally a property map. Expressing them requires the product of these choices,

so a hand-built vocabulary grows sub-linearly against a space that grows multiplicatively—which is why the dominant eval family (50.8%) is precisely the one our primitives cannot reach.

The hypothesis is falsifiable, and three predictions hold. (P1) Extending the vocabulary raises hits only inside the extended family's competence: B moves uniform recolouring $1 \rightarrow 11$ and object-level conditional $7 \rightarrow 8$; C reaches 12 and 11, while shape-changing (18) and the other 177 inputs never move. (P2) More search cannot substitute for vocabulary: ~318 chains already validate nothing on eval. (P3) Heterogeneous portfolios pay off by exactly $|H_B \setminus H_A|$: measured +11 hits, that bound. The practical consequence: instrument *coverage per family* before scaling search or model size—a cheap measurement that predicts a symbolic solver's ceiling far better than its training accuracy. Because the taxonomy is defined on input–output relations rather than on any particular solver, the measured shift is a property of the benchmark, not of our implementation.

6. Limitations and negative result

We report no improvement on the public evaluation split: **0.000% (0/167)** for A, B, C, their combinations, and every trivial baseline—eval coverage stays **0/120**, so portfolio gains cannot transfer. This is a capability boundary, not a tuning failure: the families our vocabularies cover cheaply are absent from eval by construction. We used the eval split once, as its guidance requires; our claims concern *mechanism*, not accuracy.

7. Reproducibility

Runs on CPU in seconds. The submission comes from a Kaggle notebook whose embedded engine is byte-checked against the repository source at build time; 36 self-tests and a full dry-run pass; one script re-verifies every headline number. The released notebook integrates both alternative classes, each switchable for ablation (`enable_object_family`, `enable_context_family`). Data: official ARC-AGI-2 release (Apache-2.0); code MIT-0; no weights, no external data, no network.

Links. Code: `[repository URL]`. Notebook: `[Kaggle notebook URL]`. This writeup is linked to Kaggle code submission `[submission ID]`.

Figure 1 (cover) shows the four headline measurements; **Figure 2** details the coverage curve.

Appendix: Links

- Code (MIT-0): `[repository URL]`
- Kaggle notebook (public): `[Kaggle notebook URL]`
- Linked Kaggle code submission: `[submission ID]`
- Figures: `cover.png` (Figure 1, cover image — required), `coverage_curve.png` (Figure 2)
- Data: official ARC-AGI-2 release, Apache-2.0.
- Reproduction: `python work/arc_w1/reproduce_all.py` re-verifies every number above (15/15 checks).

Appendix: Author background

Solo entrant with a background in applied machine learning and software engineering. This project was run as a deliberately bounded study: no GPU cluster, no pretrained weights, no network at inference—only

the official ARC-AGI-2 data, a CPU, and a fixed time budget. The goal was to find out *why* a classical program-search baseline collapses on ARC-AGI-2 before attempting to scale anything up.